

# Cambridge (CIE) IGCSE Computer Science

## Paper 2: Algorithms, Programming and Logic (Set A)

**Tuesday 6 May 2025**

**Morning (Time: 1 hour 45 minutes)**

Total marks

**/ 76**

### Instructions

- Try to complete this mock exam paper in one sitting, under exam conditions. Use all the time available and check your answers to each question at the end before submitting.
- Remember this is PRACTICE. Mistakes are fine and will help you improve in time for the real exam - just do your best.
- The total mark for this paper is 75.
- The number of marks for each question or part question is shown in brackets [ ].
- No marks will be awarded for using brand names of software packages or hardware.
- Calculators must not be used in this paper.

**Scan here to mark your mock exam**

or visit the mock exams landing page for this course



**1 (a) Tick (✓) one box** to identify the first stage of the program development life cycle.

**A.** Analysis

**B.** Coding

**C.** Design

**D.** Testing

**(1 mark)**

**(b) Identify three** different ways that the design of a solution to a problem can be presented.

---

---

---

**(3 marks)**

2 (a) Tick (✓) one box in each row to identify if the statement is about **validation**, **verification** or **both**.

| Statement                                                                      | Validation<br>(✓) | Verification<br>(✓) | Both<br>(✓) |
|--------------------------------------------------------------------------------|-------------------|---------------------|-------------|
| Entering the data twice to check if both entries are the same.                 |                   |                     |             |
| Automatically checking that only numeric data has been entered.                |                   |                     |             |
| Checking data entered into a computer system before it is stored or processed. |                   |                     |             |
| Visually checking that no errors have been introduced during data entry.       |                   |                     |             |

.....

.....

.....

.....

(4 marks)

(b) Give one piece of **normal** test data and one piece of **erroneous** test data that could be used to validate the input of an email address.

State the reason for your choice in each case.

.....

.....

---

---

(4 marks)

(c) The pseudocode represents an algorithm.

The pre-defined function **DIV** gives the value of the result of integer division. For example,  $Y = 9 \text{ DIV } 4$  gives the value  $Y = 2$

The pre-defined function **MOD** gives the value of the remainder of integer division. For example,  $R = 9 \text{ MOD } 4$  gives the value  $R = 1$

```
First ← 0
Last ← 0
INPUT Limit
FOR Counter ← 1 TO Limit
    INPUT Value
    IF Value >= 100
        THEN
            IF Value < 1000
                THEN
                    First ← Value DIV 100
                    Last ← Value MOD 10
                    IF First = Last
                        THEN
                            OUTPUT Value
                        ENDIF
                    ENDIF
                ENDIF
            ENDIF
        ENDIF
    NEXT Counter
```

**Complete the trace table** for the algorithm using this input data:

8, 66, 606, 6226, 8448, 642, 747, 77, 121

| Counter | Value | First | Last | Limit | OUTPUT |
|---------|-------|-------|------|-------|--------|
| -       | -     | -     | -    | -     | -      |
| -       | -     | -     | -    | -     | -      |
| -       | -     | -     | -    | -     | -      |
| -       | -     | -     | -    | -     | -      |
| -       | -     | -     | -    | -     | -      |
| -       | -     | -     | -    | -     | -      |
| -       | -     | -     | -    | -     | -      |
| -       | -     | -     | -    | -     | -      |
| -       | -     | -     | -    | -     | -      |

.....

.....

.....

.....

.....

.....

(6 marks)

3 (a) **Describe** counting in the context of algorithms.

---

---

**(2 marks)**

(b) **Write pseudocode** to count the number of times a user inputs a positive number, stopping when the user inputs a negative number.

---

---

---

**(3 marks)**

- 4 (a)** An algorithm has been written in pseudocode to allow some numbers to be input. All the positive numbers that are input are totalled and this total is output at the end. An input of 0 stops the algorithm.

```
01 Exit ← 1
02 WHILE Exit <> 0 DO
03   INPUT Number
04   IF Number < 0
05     THEN
06     Total ← Total + Number
07   ELSE
08     IF Number = 0
09     THEN
10       Exit ← 1
11     ENDIF
12   ENDIF
13 ENDIF
14 OUTPUT "The total value of your numbers is ", Number
```

**Identify the four errors** in the pseudocode and **suggest a correction** for each error.

---

---

---

---

**(4 marks)**

- (b) An algorithm has been written in pseudocode to calculate a check digit for a four-digit number. The algorithm then outputs the five-digit number including the check digit. The algorithm stops when -1 is input as the fourth digit.

```
01 Flag FALSE
02 REPEAT
03   Total 0
04   FOR Counter 1 TO 4
05     OUTPUT "Enter a digit ", Counter
06     INPUT Number[Counter]
07     Total Total + Number * Counter
08     IF Number[Counter] = 0
09       THEN
10         Flag TRUE
11     ENDIF
12   NEXT Counter
13   IF NOT Flag
14     THEN
15       Number[5] MOD(Total, 10)
16       FOR Counter 0 TO 5
17         OUTPUT Number[Counter]
18       NEXT
19     ENDIF
20 UNTIL Flag
```

**Identify the three errors** in the pseudocode and **suggest a correction** for each error.

---

---

---

(3 marks)



**5 (a)** Tick (✓) **one box in each row** to identify the most appropriate data type for each description. Only one tick (✓) per column.

| Description                           | Data type |      |         |      |        |
|---------------------------------------|-----------|------|---------|------|--------|
|                                       | Boolean   | Char | Integer | Real | String |
| a single character from the keyboard  |           |      |         |      |        |
| multiple characters from the keyboard |           |      |         |      |        |
| only one of two possible values       |           |      |         |      |        |
| only whole numbers                    |           |      |         |      |        |
| any number                            |           |      |         |      |        |

---



---



---



---

**(4 marks)**

**(b)** The variables X, Y and Z are used to store data in a program:

- X stores a string
- Y stores a position in the string (e.g. 2)
- Z stores the number of characters in the string.

**Write pseudocode statements** to declare the variables X, Y and Z.

---

---

---

(3 marks)

- (c) An algorithm has been written in pseudocode to calculate a check digit for a four-digit number. The algorithm then outputs the five-digit number including the check digit. The algorithm stops when -1 is input as the fourth digit.

```
01 Flag FALSE
02 REPEAT
03   Total 0
04   FOR Counter 1 TO 4
05     OUTPUT "Enter a digit ", Counter
06     INPUT Number[Counter]
07     Total Total + Number * Counter
08     IF Number[Counter] = 0
09       THEN
10         Flag TRUE
11     ENDIF
12   NEXT Counter
13   IF NOT Flag
14     THEN
15       Number[5] MOD(Total, 10)
16       FOR Counter 0 TO 5
17         OUTPUT Number[Counter]
18       NEXT
19     ENDIF
20 UNTIL Flag
```

**Give** the line number(s) for the statements showing:

- **Totalling**
  - **Count-controlled loop**
  - **Post-condition loop**
- 
-

---

(3 marks)

(d) **Describe** the difference between a **local** variable and a **global** variable.

---

---

(2 marks)

**6 (a)** Tick (✓) **one box** to show the name of the data structure used to store a collection of data of the same data type.

- A.** Array
- B.** Constant
- C.** Function
- D.** Variable

**(1 mark)**

**(b) Describe** a one-dimensional array.

Include **an example** of an array declaration.

---

---

---

**(3 marks)**

7 (a) Consider this logic expression.

$$Z = (\text{NOT } A \text{ OR } B) \text{ AND } (B \text{ XOR } C)$$

Draw a logic circuit for this logic expression.

Each logic gate must have a maximum of **two** inputs.

Do **not** simplify this logic expression.



---

---

---

---

(4 marks)

(b) Complete the truth table from the given logic expression.

| A | B | C | Working space | X |
|---|---|---|---------------|---|
| 0 | 0 | 0 |               |   |
| 0 | 0 | 1 |               |   |
| 0 | 1 | 0 |               |   |
| 0 | 1 | 1 |               |   |
| 1 | 0 | 0 |               |   |
| 1 | 0 | 1 |               |   |
| 1 | 1 | 0 |               |   |
| 1 | 1 | 1 |               |   |

.....

.....

.....

.....

(4 marks)

8 (a) **Describe** what a *field* is in a database.

**Provide an example** of a field in a student database

---

---

(2 marks)

(b) A music streaming service has a new database table named Songs to store details of songs available for streaming. The table contains the fields:

- **SongNumber** – the catalogue number, for example AG123
- **Title** – the title of the song
- **Author** – the name of the song writer(s)
- **Singer** – the name of the singer(s)
- **Genre** – the type of music, for example rock
- **Minutes** – the length of the song in minutes, for example 3.75
- **Recorded** – the date the song was recorded.

**Identify** the field that will be the most appropriate primary key for this table.

---

(1 mark)

(c) **Complete the table** to identify the most appropriate data type for the fields in **Songs**

| Field      | Data type |
|------------|-----------|
| SongNumber |           |
| Title      |           |
| Recorded   |           |
| Minutes    |           |

(2 marks)



9 (a) **Describe** what the algorithm represented by the flowchart in **Fig. 1** is doing.

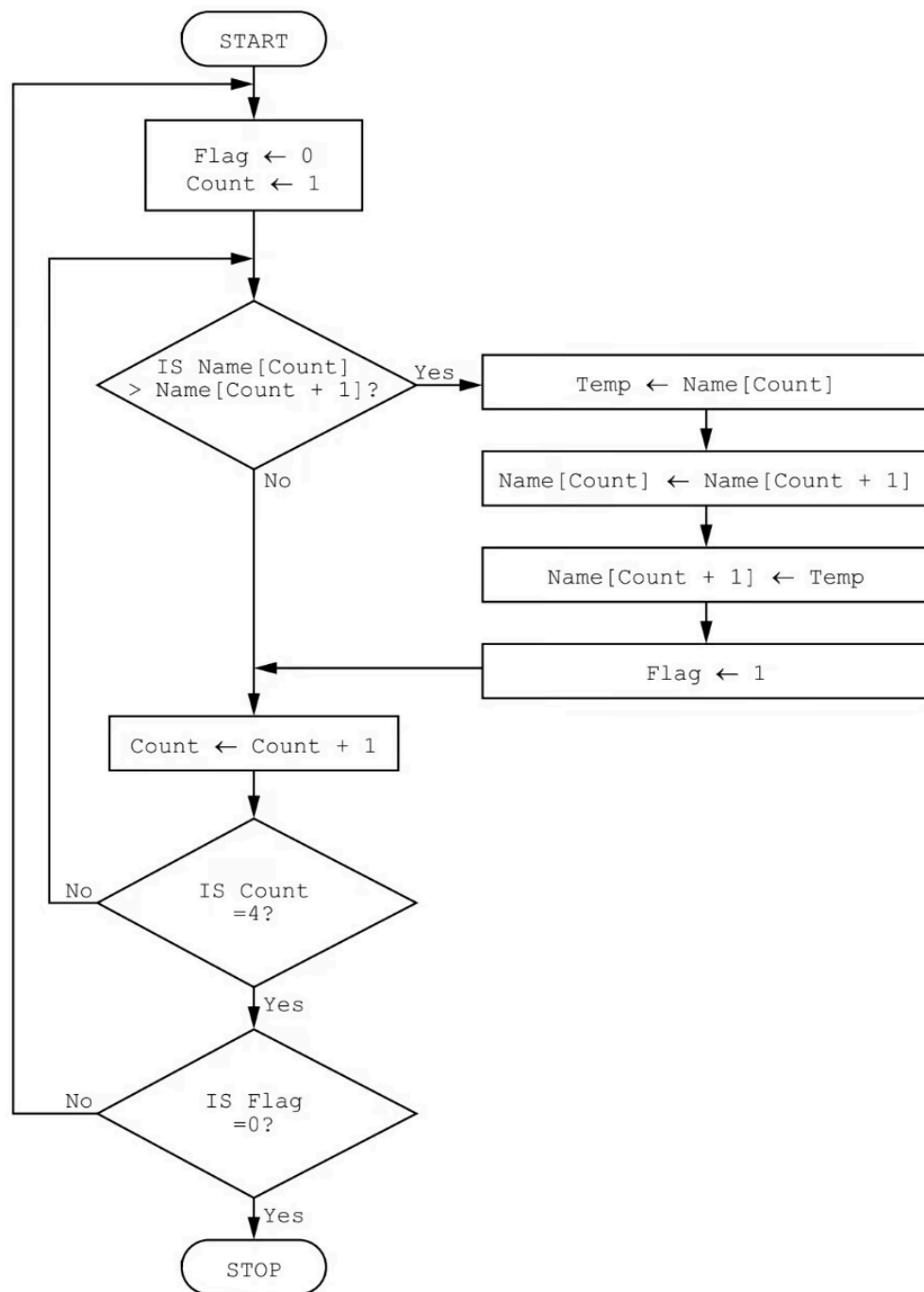


Fig.1

(2 marks)

- (b) The one-dimensional (1D) array `StudentName[]` contains the names of students in a class. The two-dimensional (2D) array `StudentMark[]` contains the mark for each subject, for each student. The position of each student's data in the two arrays is the same, for example, the student in position 10 in `StudentName[]` and `StudentMark[]` is the same.

The variable `ClassSize` contains the number of students in the class. The variable `SubjectNo` contains the number of subjects studied. All students study the same number of subjects.

The arrays and variables have already been set up and the data stored.

Students are awarded a grade based on their average mark.

| Average mark                                 | Grade awarded |
|----------------------------------------------|---------------|
| greater than or equal to 70                  | distinction   |
| greater than or equal to 55 and less than 70 | merit         |
| greater than or equal to 40 and less than 55 | pass          |
| less than 40                                 | fail          |

**Write a program** that meets the following requirements:

- calculates the combined total mark for each student for all their subjects
- calculates the average mark for each student for all their subjects, rounded to the nearest whole number
- outputs for each student:
  - **name**
  - **combined total mark**
  - **average mark**

- **grade awarded**
- calculates, stores and outputs the number of distinctions, merits, passes and fails for the whole class.

You must use **pseudocode or program code** and **add comments** to explain how your code works.

You **do not** need to initialise the data in the array.

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

**(15 marks)**